

# SENPROTECT - Backend Django

---

Ce dossier contient l'API Django de SENPROTECT. La configuration du projet est centralisee dans `config/`, tandis que les fonctionnalites metier vivent dans `apps/`.

## 1. Arborescence cible

```
back-end/
├── .env.example           # Variables a copier dans .env
├── .gitignore
├── manage.py             # Point d'entree Django
├── requirements.txt      # Dependances Python du projet
├── config/               # Configuration globale Django
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── apps/                 # Applications metier
│   ├── accounts/
│   ├── agents/
│   ├── audit/
│   ├── clients/
│   ├── contracts/
│   ├── documents/
│   └── notifications/
├── media/                # Fichiers envoyes, non versionnes
└── staticfiles/          # Fichiers collectes, non versionnes
```

Le dossier `.venv/` est cree localement a la racine de `back-end/` et ne doit jamais etre versionne.

## 2. Prerequis macOS

Installer les outils systeme une seule fois :

```
brew install python postgresql
```

Demarrer PostgreSQL :

```
brew services start postgresql
```

Verifier les versions :

```
python3 --version  
psql --version
```

Les paquets installes globalement ne sont pas utilises directement par le projet. Le projet installe ses propres dependances dans `.venv`, ce qui evite les conflits entre projets.

### 3. Creer et activer l'environnement Python

Depuis le dossier `back-end/` :

```
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt
```

Verifier que le bon Python est actif :

```
which python  
python -m pip --version  
python -c "import django; print(django.get_version())"
```

`which python` doit retourner un chemin qui se termine par `back-end/.venv/bin/python`.

A chaque nouvelle session de terminal :

```
cd chemin/vers/SENPROTECT-APP/back-end  
source .venv/bin/activate
```

### 4. Configurer les variables d'environnement

Creer le fichier local `.env` :

```
cp .env.example .env
```

Puis renseigner au minimum :

```
DEBUG=True  
SECRET_KEY=une-cle-secrete-de-developpement  
ALLOWED_HOSTS=localhost,127.0.0.1  
DB_NAME=senprotect  
DB_USER=postgres
```

```
DB_PASSWORD=votre-mot-de-passe-postgres
DB_HOST=localhost
DB_PORT=5432
```

Ne jamais committer `.env`. Le fichier `.env.example` peut être partagé car il ne contient aucun secret.

## 5. Créer la base PostgreSQL

Créer la base si elle n'existe pas :

```
createdb senprotect
```

Si votre installation PostgreSQL utilise un autre utilisateur ou port, adaptez `DB_USER`, `DB_PASSWORD` et `DB_PORT` dans `.env`.

Tester la connexion :

```
psql -h localhost -U postgres -d senprotect
```

Quitter `psql` avec `\\q`.

## 6. Initialiser Django

Toujours avec `.venv` active :

```
python manage.py check
python manage.py makemigrations
python manage.py migrate
```

`check` valide la configuration. `makemigrations` crée les migrations des modèles que vous ajoutez. `migrate` applique les migrations dans PostgreSQL.

Créer un compte administrateur :

```
python manage.py createsuperuser
```

Lancer le serveur :

```
python manage.py runserver
```

Administration : <http://127.0.0.1:8000/admin/>

## 7. Ajouter une fonctionnalite

Pour chaque fonctionnalite metier :

1. Ajouter ou modifier le modele dans `apps/<nom_app>/models.py`.
2. Ajouter les vues et endpoints dans cette application.
3. Ajouter les routes dans `config/urls.py` ou dans un fichier `urls.py` propre a l'application.
4. Executer `python manage.py makemigrations`.
5. Executer `python manage.py migrate`.
6. Executer `python manage.py check` et les tests.

Les applications sont referencees avec leur chemin complet, par exemple

`apps.accounts.apps.AccountsConfig`. Cela evite les erreurs d'import du type `No module named 'accounts'`.

## 8. Commandes de diagnostic

```
which python
python -m pip show Django
python manage.py check
python manage.py showmigrations
python manage.py dbshell
```

Si Django est introuvable, l'environnement n'est pas active ou les dependances n'ont pas ete installees :

```
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Si `config.settings` est introuvable, la commande doit etre lancee depuis `back-end/` et `manage.py` doit contenir :

```
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "config.settings")
```

- Lancement

```
cd back-end source .venv/bin/activate python manage.py runserver
```

## 9. Production

Avant un deploiement :

- definir `DEBUG=False` ;
- utiliser une vraie `SECRET_KEY` ;
- remplacer `CORS_ALLOW_ALL_ORIGINS` par une liste de domaines autorises ;
- renseigner `ALLOWED_HOSTS` ;

- executer `python manage.py collectstatic` ;
- utiliser un serveur WSGI ou ASGI configure ;
- ne jamais exposer `.env` ni les mots de passe de la base.